BESPIN GLOBAL
an e& enterprise company

© Bespin Confidential

| | |
|---|---|
| **Department** | Data Department |
| | |
| **Description** | This case studies evaluates practical Python/data skills, SQL knowledge, database concepts (backup/restore/migration), and ability to research & justify cloud data services. |
| **Case Study 1** | **Database & Migration Theory:**<br><br>**Case Study Definition**<br>A financial services company is running multiple database engines — MySQL for customer transactions, PostgreSQL for analytics, and Oracle for legacy applications. As the company prepares to migrate its infrastructure to AWS cloud, the IT team faces several challenges: ensuring business continuity during migration, understanding how different database engines behave in the cloud, and preparing robust backup and restore strategies to safeguard critical customer data against system failures. This case focuses on evaluating different database platforms, defining a reliable point-in-time recovery approach, and preparing a structured checklist to ensure a smooth and secure migration journey.<br>**Purpose / Objectives**<br>– Compare the database engines (MySQL, PostgreSQL, SQL Server, Oracle, MongoDB) in terms of components, use cases, and migration readiness.<br>– Design a backup and restore startigy for one database engine that enables point-in-time recovery (PITR) after a system failures incident with data lose less that 1 hour.<br><br>**Expected Deliverables / Outcomes**<br><br>Prepare a PDF file to cover the following:<br><br>● **Database Engine Comparison Report**: clearly written comparison of three selected engines, including use cases, three database features , one advantage, and one limitation for migrate that database engine to AWS cloud.<br>● **Backup & Restore Plan:** a documented backup strategy for one database engine that grantee point-in-time recovery with data lost less than 1 hour including type of backups used. sample commands/pseudocode<br><br>● **Data Center Database to AWS cloud Mapping** : table or report mapping each engine (MySQL, PostgreSQL, SQL Server, Oracle , MongoDB) to its AWS equivalent(s), with a short justification and advantage of migrating it to AWS cloud |
| **Case Study 2** | **Practical Data Processing**<br><br>**Case Study Definition**<br>● A retail client exports a daily **transactions_input.csv** feed from multiple stores. The feed is noisy: some rows arrive without a transaction ID and some have malformed timestamps. You've been asked to build a small, reliable Python ETL that reads the file , **drops only rows lacking a transaction ID**, produces clean **transactions_cleaned.csv**, highlights problematic records, and summarizes top customers for business review.<br><br>**Purpose / Objectives**<br>● Implement a Python ETL that is reproducible and easy to run on a laptop.<br>● Apply a single cleaning rule (remove rows with empty transaction_id)<br>● Produce three business-ready CSV outputs from the **transactions_cleaned.csv** file .<br>● **Perform basic aggregations** from the cleaned dataset**.** |

**Expected Deliverables / Outcomes**

**Input :** *transactions_input.csv* (provided by us)

**Required outputs :**

1- ***transactions_cleaned.csv* file:**
   - Contains only rows where transaction_id **is non-empty** (not ""/null).
   - Columns preserved exactly as in the input: transaction_id, customer_id, amount, currency, transaction_ts, product_id

2- ***transactions_empty.csv* file:**
   - Contains only rows where transaction_id **is empty.**
   - Columns preserved exactly as in the input: transaction_id, customer_id, amount, currency, transaction_ts, product_id

3- ***monthly_sales.csv* file** (derived from transactions_cleaned.csv**)**
   - Shows the company's total sales amount per month
   - **Columns:**
     - month (derived from transaction_date, format YYYY-MM)
     - total_amount (sum of all transactions in that month)
     - total_transactions (count of transactions in that month)

4- ***top_10_customers_overall.csv*** (from transactions_cleaned.csv**)**
   - Lists top 10 customers who contributed the most revenue overall.
   - **Columns:**
     - customer_id
     - total_amount (sum of all purchases by that customer)
     - total_transactions (count of that customer's transactions)

Notes :

- Use **Python 3.8+** (any approach: standard libraries or pandas).
- Provide a **README.txt** with steps for **macOS/Linux or Windows**, depending on what you used, so we can easily run and validate your code**.**
- The goal of this case study is to evaluate your ability to think and code with Python for data problems. A working script is required
- Make the code readable with proper comments

BESPIN GLOBAL

an enterprise company